

Assignment: Joins & its Types — Hands-on & Scenario-Based Practice

Note: Work inside `college_db.academics`. All questions build on the tables and data you created/populated in Previous assignments (students, departments, courses, enrollments). Write and test every statement in pgAdmin's Query Tool. For scenario questions, actually run the steps and observe the real result before answering.

Setup Reference

Confirm the following tables exist and contain consistent data (adjust if needed):

Table	Purpose / Key Columns
academics.departments	dept_id (PK), dept_name
academics.students	student_id (PK), full_name, email, dept_id (FK), cgpa, age
academics.courses	course_id (PK), course_name, dept_id (FK), credits
academics.enrollments	student_id + course_id (composite PK), enroll_date, grade

If any table is empty or missing required columns, populate at least:

- 4+ departments, 8+ students, 6+ courses, 12+ enrollments
- At least one student with no enrollments
- At least one course with no enrollments
- At least one department with no students

Q1. Basic INNER JOIN

Write a query that lists every student's full_name together with their department name. Use an INNER JOIN. Order by dept_name, then full_name.

-- Write your SELECT statement here:

```
SELECT
    s.full_name,
    d.dept_name FROM academics.students s
INNER JOIN academics.departments d
ON s.dept_id = d.dept_id
ORDER BY d.dept_name, s.full_name;
```

Q2. LEFT JOIN — Students without a Department

Some students may still have NULL dept_id. Write a LEFT JOIN query that returns all students and their department name (NULL if unassigned). Then add a WHERE clause to show only the students who have no department assigned.

-- Write both SELECT statements here:

```
SELECT
    s.student_id,
    s.full_name
FROM academics.students s
LEFT JOIN academics.departments d
ON s.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Q3. Multi-table INNER JOIN

Produce a report showing: student full_name, course_name, grade, and department name of the student. Join students → enrollments → courses → departments. Only include graded enrollments (grade IS NOT NULL). Order by full_name, course_name.

-- Write your *SELECT* statement here:

```
SELECT
    s.full_name,
    c.course_name,
    e.grade,
    d.dept_name
FROM academics.students s
INNER JOIN academics.enrollments e
ON s.student_id = e.student_id
INNER JOIN academics.courses c
ON e.course_id = c.course_id
INNER JOIN academics.departments d
ON s.dept_id = d.dept_id
WHERE e.grade IS NOT NULL
ORDER BY s.full_name, c.course_name;
```

Q4. Find Students Not Enrolled in Any Course

The college wants a list of students who have not enrolled in even a single course. Write a LEFT JOIN (anti-join pattern) that returns student_id and full_name of such students.

-- Write your *SELECT* statement here:

```
SELECT
    s.student_id,
    s.full_name
FROM academics.students s
LEFT JOIN academics.enrollments e
ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Q5. Courses Never Taken

Similarly, list all courses that have zero enrollments (course_id and course_name). Use LEFT JOIN + IS NULL pattern.

-- Write your *SELECT* statement here:

```
SELECT
    c.course_id,
    c.course_name
FROM academics.courses c
LEFT JOIN academics.enrollments e
ON c.course_id = e.course_id
```

WHERE e.course_id IS NULL;

Q6. Department-wise Student Count

Write a query that shows every department (even those with zero students) along with the count of students belonging to it. Use LEFT JOIN + COUNT + GROUP BY. Order by student count descending.

-- Write your SELECT statement here:

```
SELECT
    d.dept_name,
    COUNT(s.student_id) AS student_count
FROM academics.departments d
LEFT JOIN academics.students s
ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY student_count DESC;
```

Q7. RIGHT JOIN Practice

Using a RIGHT JOIN, list every course along with the names of students enrolled in it. Courses with no enrollments should still appear (student name will be NULL).

-- Write your SELECT statement here:

```
SELECT
    c.course_name,
    s.full_name
FROM academics.enrollments e
RIGHT JOIN academics.courses c
ON e.course_id = c.course_id
LEFT JOIN academics.students s
ON e.student_id = s.student_id
ORDER BY c.course_name;
```

Q8. FULL OUTER JOIN — Orphans on Both Sides

Write a FULL OUTER JOIN between students and enrollments that reveals:

- Students with no enrollments, and
- Enrollments that somehow reference a non-existent student (should not happen if FKs are correct, but useful for data-quality checks).

Return student_id, full_name, course_id, grade. Filter to rows where either side is missing.

-- Write your SELECT statement here:

```
SELECT
    s.student_id,
    s.full_name,
    e.course_id,
    e.grade
FROM academics.students s
FULL OUTER JOIN academics.enrollments e
ON s.student_id = e.student_id
```

```
WHERE s.student_id IS NULL  
OR e.student_id IS NULL;
```

Q9. SELF JOIN — Employee Hierarchy

Using the employees table from Day 3 Q12 (or create it if missing: emp_id, emp_name, manager_id), write a self-join that displays each employee next to their manager's name. Include employees who have no manager (top-level). Use LEFT JOIN.

-- Write your CREATE (if needed), INSERT (if needed), and SELECT here:

```
CREATE TABLE academics.employees  
(  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    manager_id INT,  
    CONSTRAINT fk_manager  
        FOREIGN KEY(manager_id)  
        REFERENCES academics.employees(emp_id)  
);  
INSERT INTO academics.employees  
VALUES  
(1,'Principal',NULL),  
(2,'Vice Principal',1),  
(3,'Professor',2),  
(4,'Assistant Professor',3);  
  
SELECT  
    e.emp_name AS employee,  
    m.emp_name AS manager  
FROM academics.employees e  
LEFT JOIN academics.employees m  
ON e.manager_id = m.emp_id;
```

Q10. Students and Their Course Load

For each student, show: full_name, number of courses enrolled, and average grade (treat grade as text for now — just COUNT the graded enrollments). Include students with zero enrollments (show 0). Hint: LEFT JOIN + GROUP BY + COUNT.

-- Write your SELECT statement here:

```
SELECT  
    s.full_name,  
    COUNT(e.course_id) AS total_courses,  
    COUNT(e.grade) AS graded_courses  
FROM academics.students s  
LEFT JOIN academics.enrollments e  
ON s.student_id = e.student_id  
GROUP BY s.student_id, s.full_name  
ORDER BY s.full_name;
```

Q11. JOIN ... USING vs ON

(a) Rewrite the query from Q1 using JOIN ... USING (dept_id) instead of ON.

(b) In 1-2 sentences explain when USING is convenient and when it can become risky.

-- (a) Statement and (b) Explanation:

A) *SELECT*

full_name,

dept_name

FROM academics.students

INNER JOIN academics.departments

USING (dept_id)

ORDER BY dept_name, full_name;

B) *using* is convenient when both tables have a column with the same name because you don't need to write `table1.column = table2.column`

Q12. Complex Reporting Query

Produce a single result set that answers: "Which departments have students enrolled in courses that belong to a different department?" (cross-department enrollments). Show: student_name, student_dept, course_name, course_dept. Only include rows where the two department names differ.

-- Write your *SELECT* statement here:

SELECT

s.full_name AS student_name, sd.dept_name AS student_department,

c.course_name,

cd.dept_name AS course_department FROM academics.students s

INNER JOIN academics.departments sd

ON s.dept_id = sd.dept_id

INNER JOIN academics.enrollments e

ON s.student_id = e.student_id

INNER JOIN academics.courses c

ON e.course_id = c.course_id

INNER JOIN academics.departments cd

ON c.dept_id = cd.dept_id

WHERE sd.dept_name != cd.dept_name;